MEDAC
Instituto Oficial de Formación Profesional

Trabajo voluntario de subida de notas

Agustín Borreguero Castro

Índice

1. Introducción 3
2. Arquitectura del Sistema 3
3. Gestión de Procesos y Concurrencia 4
   3.1 Hilos de Consulta 4
   3.2 Sincronización 5
4. Gestión de Urgencias 5
5. Asignación Dinámica de Veterinarios 5
6. Comunicación en Red (Sockets) 6
7. Base de Datos 6
   7.1 Tabla Pacientes 6
   7.2 Tabla Atenciones 7
8. Pruebas de Concurrencia 7
9. Problemas Encontrados y Soluciones 8
10. Posibles Mejoras Futuras 8
11. Conclusión 9

1. Introducción

El objetivo de este proyecto es desarrollar un sistema de gestión de procesos para una clínica veterinaria aplicando los contenidos trabajados en la asignatura de Programación de Servicios y Procesos. La idea principal es simular el funcionamiento real de una clínica donde varios pacientes llegan al mismo tiempo y deben ser atendidos de manera organizada y eficiente.

La clínica dispone de dos consultas que pueden funcionar simultáneamente, un sistema de urgencias para priorizar casos graves y tres veterinarios que atienden a los pacientes. El sistema desarrollado debe ser capaz de gestionar la llegada de pacientes, diferenciar entre consultas normales y urgencias, asignar automáticamente el veterinario más adecuado según su carga de trabajo, permitir comunicación interna mediante red y registrar toda la información en una base de datos SQL.

Con este proyecto se busca aplicar de forma práctica conceptos como hilos, sincronización, comunicación entre procesos y persistencia de datos.

2. Arquitectura del Sistema

El sistema está dividido en varios módulos con responsabilidades diferenciadas para mantener una estructura clara y organizada:

- **Modelos (Paciente y Veterinario):** representan las entidades principales del sistema. El paciente contiene información como nombre, tipo y prioridad, mientras que el veterinario mantiene su carga actual de trabajo.
- **Clase Clínica:** actúa como núcleo del sistema. Gestiona las colas de pacientes, controla la sincronización entre hilos y coordina la asignación de veterinarios.
- **Hilos de Consulta:** simulan las dos consultas físicas de la clínica. Cada hilo ejecuta de forma independiente el proceso de atención.
- **Servidor de red:** permite enviar notificaciones en tiempo real a clientes conectados mediante sockets.

- **Base de datos SQLite:** almacena de forma persistente los pacientes y las atenciones realizadas.

Esta separación facilita el mantenimiento del código y permite ampliar el sistema en el futuro.

3. Gestión de Procesos y Concurrencia

El aspecto más importante del proyecto es la gestión de procesos concurrentes, ya que existen varias consultas funcionando al mismo tiempo y compartiendo recursos.

Se han utilizado hilos para simular las consultas y mecanismos de sincronización para evitar problemas como condiciones de carrera o inconsistencias en los datos. De esta forma, el sistema puede atender pacientes simultáneamente sin perder información ni producir errores.

3.1 Hilos de Consulta

Cada consulta se implementa como un hilo independiente utilizando “extends Thread". Existen dos hilos activos que representan las dos consultas disponibles en la clínica.

Cada hilo realiza el siguiente flujo:

1. Obtiene el siguiente paciente disponible.
2. Selecciona el veterinario con menor carga.
3. Registra el inicio de la atención en la base de datos.
4. Simula el tiempo de atención mediante “Thread.sleep()”.
5. Registra el fin de la atención.
6. Reduce la carga del veterinario.

Gracias a este diseño, el sistema puede procesar pacientes en paralelo, simulando un entorno real de trabajo.

3.2 Sincronización

Para evitar conflictos al acceder a recursos compartidos (listas de pacientes y lista de veterinarios), se ha utilizado el modificador **"synchronized"** en los métodos críticos.

Además, se ha implementado el uso de:

- **wait():** para que los hilos esperen cuando no hay pacientes disponibles.
- **notifyAll():** para despertar los hilos cuando se registra un nuevo paciente.

Esto garantiza que el acceso a las listas sea seguro, evitando errores de concurrencia.

4. Gestión de Urgencias

El sistema diferencia entre pacientes normales y urgentes mediante dos listas separadas. Esto permite aplicar una lógica de priorización clara.

Cuando una consulta solicita un paciente:

- Si existen urgencias, se selecciona la que tenga menor valor de prioridad.
- Si no hay urgencias, se atiende al siguiente paciente normal por orden de llegada.

Este sistema garantiza que los casos graves sean atendidos lo antes posible, simulando el funcionamiento real de una clínica veterinaria.

5. Asignación Dinámica de Veterinarios

Cada veterinario mantiene un contador interno que representa su carga actual de trabajo. Cuando una consulta necesita asignar un veterinario, el sistema recorre la lista y selecciona el que tenga menor carga en ese momento.

Este mecanismo implementa un balanceo básico de carga, evitando que siempre trabaje el mismo veterinario y distribuyendo el trabajo de forma más equilibrada.

6. Comunicación en Red (Sockets)

Se ha implementado un sistema de comunicación interna utilizando sockets de Java. El servidor utiliza **“ServerSocket”** para escuchar conexiones entrantes y **“Socket”** para gestionar cada cliente conectado.

Para la transmisión de datos se han empleado **“DataInputStream”** y **“DataOutputStream"**, utilizando los métodos **“writeUTF()”** y **“readUTF()”** para enviar y recibir mensajes.

Cada vez que ocurre un evento relevante (nuevo paciente, inicio o fin de atención), el servidor envía una notificación a todos los clientes conectados. Esto simula un sistema de mensajería interna que permite conocer en tiempo real el estado de la clínica.

7. Base de Datos

Se ha utilizado SQLite como base de datos embebida, conectada mediante JDBC. Esta solución permite almacenar información sin necesidad de un servidor externo.

La base de datos garantiza persistencia, es decir, que la información se mantenga incluso después de cerrar el programa.

7.1 Tabla Pacientes

La tabla pacientes almacena la información básica de cada paciente que entra en el sistema:

- id
- nombre
- tipo (consulta o urgencia)
- prioridad

Cada vez que se registra un paciente en la clínica, también se inserta automáticamente en esta tabla.

7.2 Tabla Atenciones

La tabla de atenciones permite llevar un seguimiento completo de cada atención realizada. Contiene:

- id
- paciente_id
- veterinario
- consulta
- inicio
- fin

Cuando comienza una atención se inserta un registro con la hora de inicio. Cuando finaliza, se actualiza el campo fin. Esto permite analizar tiempos de atención y tener un historial detallado.

8. Pruebas de Concurrencia

Se han realizado pruebas simulando múltiples pacientes consecutivos para comprobar el comportamiento del sistema bajo carga.

Durante las pruebas se observó que:

- Las urgencias siempre se priorizan correctamente.
- Nunca se ejecutan más de dos atenciones simultáneas.
- No se producen bloqueos ni errores de concurrencia.
- La base de datos registra correctamente cada evento.

Esto demuestra que el sistema es estable y que la sincronización está correctamente implementada.

9. Problemas Encontrados y Soluciones

Conflicto de clave primaria

Al ejecutar varias veces el programa, SQLite producía un error de clave primaria duplicada.

Se solucionó eliminando la base de datos entre ejecuciones o utilizando INSERT OR IGNORE.

Sincronización de listas

Se detectó que sin synchronized podían producirse inconsistencias al acceder a las listas desde varios hilos.

Se solucionó sincronizando los métodos críticos.

10. Posibles Mejoras Futuras

El sistema podría ampliarse con:

- Una interfaz gráfica para hacerlo más visual y fácil de usar.
- Estadísticas automáticas de tiempos de atención.
- Sistema de autenticación de usuarios.
- Implementación del patrón MVC para separar mejor lógica y presentación.

Estas mejoras permitirían convertir el proyecto en una aplicación más completa y profesional.

11. Conclusión

En este proyecto se ha desarrollado un sistema de gestión para una clínica veterinaria aplicando los principales conceptos de Programación de Servicios y Procesos. Se han implementado hilos para simular las consultas, sincronización para evitar conflictos de concurrencia y un sistema de priorización para gestionar correctamente las urgencias.

Además, se ha integrado comunicación en red mediante sockets y persistencia de datos con SQLite, permitiendo registrar pacientes y hacer seguimiento de cada atención. Las pruebas realizadas demuestran que el sistema funciona de forma concurrente sin bloqueos y mantiene la coherencia de la información.

En conjunto, el proyecto cumple los requisitos planteados y demuestra una aplicación práctica de la programación concurrente, la comunicación entre procesos y la gestión de recursos compartidos.